

Q-GEAR: Improving quantum simulation framework

Ziqing Guo*

Department of Computer Science
Texas Tech University
Lubbock, TX, USA
NERSC

Lawrence Berkeley National Laboratory
Berkeley, CA, USA
ziqguo@ttu.edu

Jan Balewski

Lawrence Berkeley National Laboratory
Berkeley, CA, USA
balewski@lbl.gov

Ziwen Pan

Department of Computer Science
Texas Tech University
Lubbock, TX, USA
ziwen.pan@ttu.edu

Abstract

The rapid execution of complex quantum circuit simulations is essential for validating theoretical algorithms, thereby facilitating their successful implementation on quantum hardware. Although mainstream CPU-based platforms for circuit simulations are well established, they tend to be slower. Conversely, the adoption of GPU platforms remains limited because of the necessity for specialized quantum simulation frameworks tailored to different hardware architectures, each requiring distinct implementation and optimization strategies. Therefore, we introduced **Q-Gear**, a platform-agnostic framework that transforms **Qiskit** quantum circuits into **Cuda-Q** kernels. By leveraging **Cuda-Q** seamless execution on GPUs, **Q-Gear** accelerates both CPU- and GPU-based simulations by two orders of magnitude and ten times, respectively, with minimal coding effort. Furthermore, **Q-Gear** leverages the **Cuda-Q** configuration to interconnect the memory of GPUs, allowing the execution of much larger circuits beyond the memory limit set by a single GPU or CPU node. Additionally, we created and deployed a Podman container and Shifter image at Perlmutter (NERSC/LBNL), both derived from an NVIDIA public image. These public NERSC containers were optimized for the Slurm job scheduler, allowing approximately 100% utilization of up to 1,024 GPUs. We present various benchmarks for **Q-Gear** to demonstrate the efficiency of our computational paradigm.

CCS Concepts

• Computer systems organization → Quantum computing.

Keywords

Quantum Circuit, High Performance Computing, Image Encoding.

ACM Reference Format:

Ziqing Guo, Jan Balewski, and Ziwen Pan. 2025. Q-GEAR: Improving quantum simulation framework. In *54th International Conference on Parallel Processing (ICPP '25)*, September 08–11, 2025, San Diego, CA, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3754598.3754608>

1 Introduction

Quantum circuit simulations (QCS) directly model the mathematical formalism of complex quantum states. Quantum methods are

*Also with NERSC, LBNL, Berkeley, CA.



This work is licensed under a Creative Commons Attribution 4.0 International License. *ICPP '25, San Diego, CA, USA*

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2074-1/25/09

<https://doi.org/10.1145/3754598.3754608>

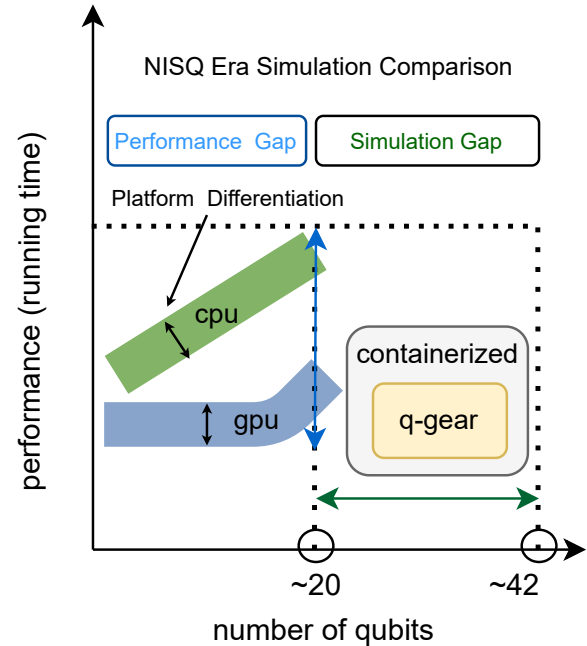


Figure 1: An overview of contemporary optimal quantum simulation techniques and our proposed method is presented. The parallelized GPU architecture demonstrates enhanced scalability and accelerated computation for extensive matrix operations compared to the CPU, as indicated by the blue arrow (with the black arrow denoting the framework differences). The green arrow signifies that our method facilitates scalability from 20 to 42 qubits.

expected to outperform classical methods in database searching [14] and factoring problems [33] because they bypass the need to explicitly represent and manipulate exponentially large state vectors, which is a limitation of classical algorithms. These advantages have led to the growing prominence of quantum methods in both fundamental research and widespread application in various domains, including cryptography [30], materials science [20], and quantum machine learning [5].

The most well-known approach to quantum computing systems involves designing a sequence of basic unitary operations known as quantum gates to transform a standard initial state into a specific target quantum state [32]. Modern QCS tackles a diverse range of tasks, including variational quantum algorithms [6, 8, 24] and

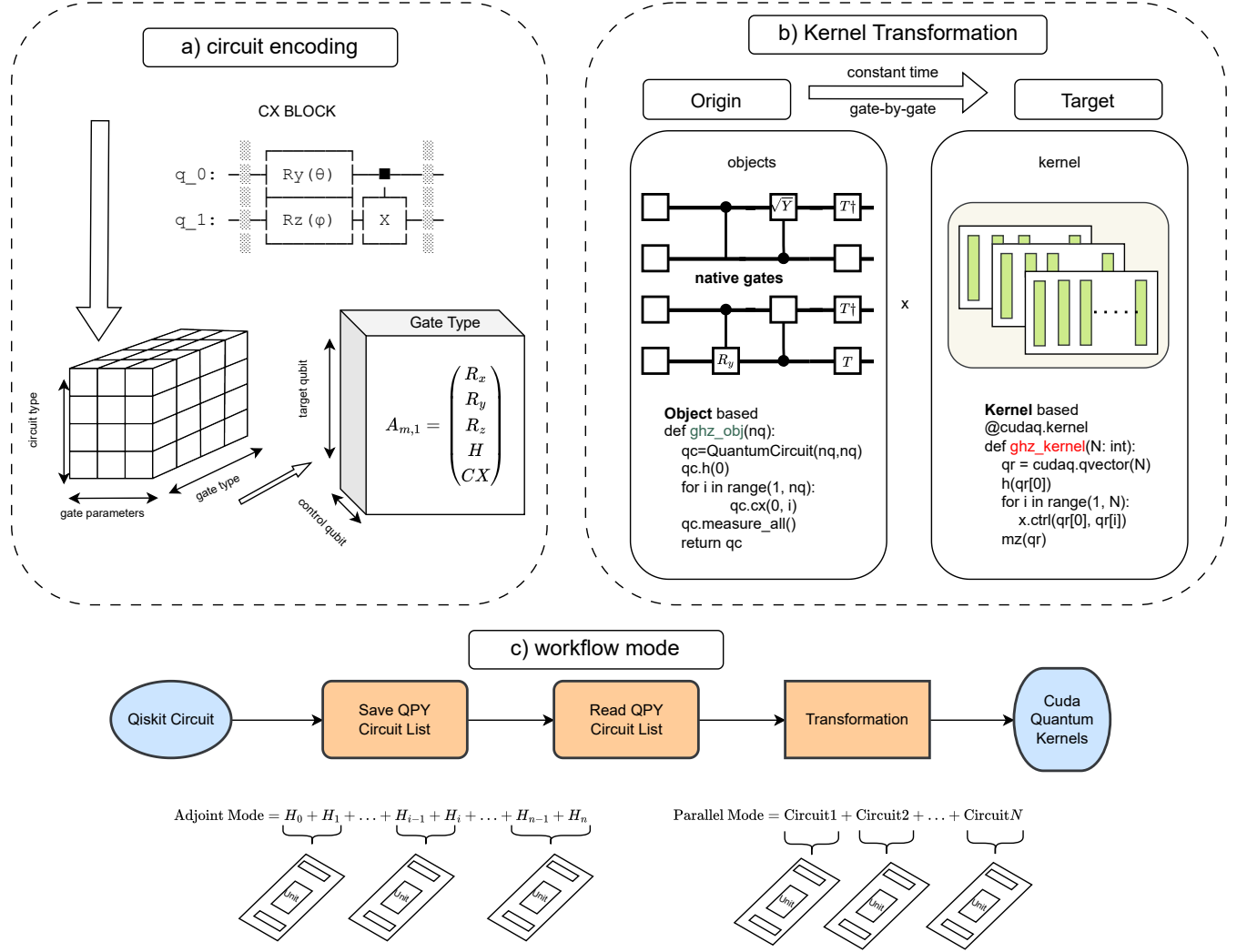


Figure 2: Summary of the Q-Gear methods and workflows. (a) The quantum circuits (exampled as a CX block) are encoded as three-dimensional tensors, with gate types ((R_x, R_y, R_z, H, CX) denoted by $A_{m,1}$) further encapsulated into tensor blocks specifying the control and target qubits. (b) These tensorial circuits are converted into kernel circuits optimized for CUDA execution within thread warps, offering $O(1)$ transformation overhead and ensuring just-in-time compilation for the GPU. (c) The final kernelized quantum circuits are prepared for simulation using the adjoint mode for multi-GPU single-circuit simulation and a parallel model for one GPU per circuit-mapped simulation. Note that our workflow is containerized into Podman which further enhances the convenience of the blocked run.

hybrid quantum-classical frameworks [4, 17]. These frameworks are supported by techniques such as unitary compilation [39], ansatz initialization [29], and circuit optimization [26]. The implementation of these tasks relies on tools such as **Qiskit** [25], **PennyLane** [1], **Cuda-Q** [21], **MPICH** [13], and **Podman** [34] containers. The key focus of these techniques is to improve the measurement fidelity using millions of shots, and to improve the scalability of quantum computations.

However, the QCS efficiency remains highly sensitive to circuit layouts, with scaling challenges becoming more pronounced as the circuit complexity increases. Specifically, as the number of qubits,

depth of the circuit, and complexity of the circuit entanglement increase, the simulation overhead increases significantly. Furthermore, IBM experts demonstrated possible integration of **Qiskit** with a V100 GPU [15]. However, this process requires the installation of **Qiskit** from the source, which is a complex and nontrivial task. Their performance benchmarks on QFT circuits (up to 22 qubits) reported speedups of less than 10x utilizing the single-float accuracy. For completeness, we refer to the benchmark [11] for evaluating the finicky tasks.

To address these QCS challenges, we introduce **Q-Gear** [40] which provides a platform-agnostic containerized running mode that

fully utilizes state-of-the-art GPU to accelerate the entire program. Therefore, with minimal coding effort, **Q-Gear** allows QCS of circuits that are significantly larger and faster than those feasible on typical CPUs, as shown in Fig. 1. Specifically, **Q-Gear** optimizes the performance by distributing workloads from native objects to CUDA kernels using the MPI framework. This enables scalability to higher qubit counts, reduces simulation runtime, and enhances efficiency, while maintaining flexibility for user adaptability. We refer Appendix A for the details of QCS.

2 Methods

In this section, we present the comprehensive methods of **Q-Gear** to encode circuit sequences into **Cuda-Q** kernels, and the architecture of the containerized workflow that fully utilizes GPU power using **Podman** [34] and Slurm jobs [18].

2.1 Circuit encoding

To accurately map the quantum circuits, we converted the saved gate list into a three-dimensional tensor comprising matrices and tensors. As shown in (a) of Fig. 2, the first dimension encodes the type of circuit, indices of the qubits, and the gate count. The second dimension stores the gate categories, control qubit indices, and target qubit indices. The third dimension captures the unified gate parameters extracted from the QPY file [16], transpiled from the native gate sets, and maps the variables into predefined tensors that remain fixed but are dynamically updated based on the input quantum circuits during encoding.

The lengths of the tensors correspond to the number of encoded circuits. Although the illustrated graph demonstrates a CX block, more complex block encoding and highly entangled scenarios [27] require larger data storage for transformation-encoding. We defer the analysis of high-dimensional data for HPC in Appendix B. To address this, we used the HDF5 [22] file format, which efficiently encodes and manages high-dimensional datasets, supports various data types (including metadata integration), and provides scalable storage for complex scientific and computational workflows. This approach ensures a constant circuit-conversion time. The details and proof of the encoding process are provided in Appendix C.

2.2 Kernel transformation

To transform circuits into CUDA kernels efficiently, **Q-Gear** directly converts **Qiskit** quantum circuits into GPU-executable kernels. Each kernel operates as a user-controllable thread, leveraging GPU parallelism to achieve a four-fold speed-up within a single node. Untransformed Qiskit circuits are targeted for conversion, which represent transpiled pulse-like gates constrained by native QPU specifications or high-level objects containing native circuit data, as shown in Fig. 2b. To map object-based circuits into kernel-based representations, we define a custom **Cuda-Q** kernel that incorporates qubits, rotation angles, and unitary operations, thereby enabling the decoding of transformed quantum circuits directly into the CUDA kernels. In parallel, this transformation fully utilizes the MPI parallel memory sharing and public-channel communication. Consequently, parameterized kernel transformations preserve the structure of the final converted circuits while maximizing computational efficiency.

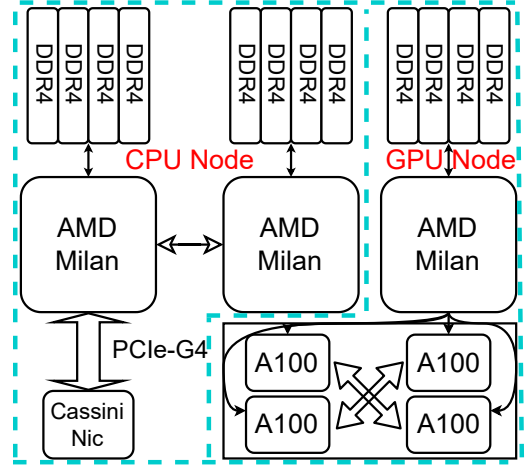


Figure 3: CPU and GPU node layouts.

2.3 Benchmarking hardware

Based on the experimental and empirical results, we evaluated **Q-Gear** using two hardware configurations: one with AMD EPYC 7763 processors (CPU node) and the other with NVIDIA A100 GPUs (GPU node) [9, 36]. The CPU node was powered by two AMD EPYC 7763 (Milan) processors, each with 64 cores and AVX2 support, delivering 39.2 GFLOPS per core (2.51 TFLOPS per socket). It features 512 GB of DDR4 memory with a bandwidth of 204.8 GB/s per CPU and is connected via an HPE Slingshot 11 NIC over PCIe 4.0, configured with four NUMA domains per socket (NPS=4). The GPU node included a single AMD EPYC 7763 (Milan) CPU with 64 cores paired with four NVIDIA A100 (Ampere) GPUs. Each GPU provides up to 2039 GB/s of memory bandwidth (80GB HBM2e) and is interconnected via four third-generation NVLinks, delivering 25 GB/s per direction per link. The node features a 256 GB DDR4 DRAM with a 204.8 GB/s CPU memory bandwidth, connected via PCIe 4.0 to the GPUs and four HPE Slingshot 11 NICs, ensuring high-performance data throughput and seamless integration of the GPU-CPU-NIC (see Fig. 3).

2.4 Pipeline

To demonstrate the generality of our transformation, we incorporated two key architectures into our framework: the QFT transformation [37, 38] and Quantum Image Representation [2, 23]. Our heterogeneous workflow maximizes GPU utilization by integrating **Podman** for portable, consistent containerized simulations and Slurm for efficient job scheduling, ensuring optimal task distribution, workload balance, and minimal idle resources. This approach achieved near-peak GPU performance in large-scale quantum circuit simulations. For larger and more complex circuits, the simulation process partitions them into distinct Hamiltonians that represent the evolution of quantum systems (see (c) in Fig. 2). These Hamiltonians are distributed across multiple hardware resources, thereby enabling efficient parallelization of the algorithm. Additionally, the parallel mode allows the simultaneous execution of multiple smaller quantum circuits on separate GPUs, significantly enhancing performance.

compared with sequential simulations [27]. Detailed examples are provided in Appendix D.

3 Results

We show that **Q-Gear** is a lightweight and efficient interface between **Qiskit** and **Cuda-Q** that accelerates quantum circuit simulations when GPUs are available without requiring the recoding of quantum circuits. By leveraging the **Cuda-Q** 'nvidia' target, **Q-Gear** enables simulations of up to 42 qubits on a cluster of 1024 GPUs with a single circuit spread over all GPUs. Below, we show the performance of **Q-Gear** for three representative cases: (1) random non-Clifford units, (2) quantum Fourier transform (QFT), and (3) **QCrack** quantum circuits that encode grayscale images.

Q-Gear simulation of non-Clifford unitaries

To demonstrate the versatility of **Q-Gear**, we compare the simulation times of randomly generated non-Clifford [10] unitaries for a preset number of qubits. The two types of random unitaries used in this benchmark are constructed from 100 or 10,000 two-qubit blocks, where each block consists of two random single-qubit rotations followed by an entangling gate (see the visualization in Fig. 2 and detailed implementation in Appendix E.1). This random unitary structure effectively models nontrivial workloads in quantum algorithms with an increasing computational complexity. We refer to these as 'short' or 'long' random unitaries.

Fig. 4a demonstrates baseline performance for circuit simulation time with a CPU-based **Qiskit** backend Aer [16], shown as dashed curves / open symbols with settings detailed in Table 1. The experiments were carried out on a Perlmutter CPU node with 512 GB of DDR4 in total and 128 compute cores¹. Solid curves and closed symbols represent simulation times for the same two types of unitaries, executed using **Q-Gear** with **Cuda-Q** on either one or four A100 GPUs, with the backend target set to 'nvidia-mgpu' [21].

For short unitaries, all available CPU RAM was exhausted at 34 qubits, as shown by the open squares. For long unitaries, which contain 100 times more entangling gates, the **Qiskit** simulation takes 100 times longer, as shown by the open circles. Both cases follow a similar exponential scaling of execution time $\sim 2^n$, where n is the number of qubits. One may anticipate that it would take approximately 24 h to simulate a single 34-qubit unitary with 10,000 CX gates on one CPU node.

We achieved a consistent 400-fold speedup in the simulation with **Q-Gear** on a single GPU using the container, as indicated by the solid squares in Fig. 4a. This is non-trivial because the same **Qiskit** circuits are exported as **NumPy** arrays in the format specified by the **Q-Gear** framework and converted to **Cuda-Q** kernels (see Section 2.2), leveraging multithreading and GPU parallelism to optimize resource utilization. This approach enables seamless integration between **Qiskit** and **Cuda-Q** backends, either within a single program or by saving **NumPy** circuits in the format **HDF5** [22] for use in a separate **Cuda-Q** program. The **Cuda-Q** simulation on a single A100 GPU with a 40 GB RAM restricts the simulable unitary to a maximum of 32 qubits.

In our solution, the **Q-Gear** framework can overcome single GPU RAM limitations by setting the **Cuda-Q** target to 'nvidia-mgpu' instead of 'nvidia', which effectively combines the memory from multiple GPUs. Fig. 4a shows solid triangles as the execution times of the same unitaries on 4 interconnected A100 GPUs. This configuration enables the simulation of up to a 34-qubit circuit, where adding only two additional qubits requires four times more memory. The 34-qubit unitary was simulated on four GPUs in 1 min, compared to 24 h for the CPU node **Qiskit** simulation. We note that the **Cuda-Q** target 'nvidia-mgpu' significantly improves the execution time for 32-qubit circuits by leveraging parallelism across four GPUs, effectively utilizing them as four quantum processing units, compared to single-GPU execution.

To demonstrate **Q-Gear**'s ability to simulate circuits with an even larger number of qubits in the Perlmutter system, we constructed an intermediate-size unitary consisting of 3,000 random entangling blocks and varied the qubit count from 30 to 42, executing them on an increasingly larger cluster of A100 GPUs, ranging from 4 to 1024. The execution times are shown in Fig. 4b, where the colors of the symbols change according to the size of the GPU cluster. It is evident that **Q-Gear** can efficiently handle such large-circuit simulations within a reasonable time of approximately 10 min, provided that a sufficient number of GPUs is available.

Interestingly, we observed that adding more GPUs did not always reduce execution time. For example, in the highlighted region in Fig. 4b, where the number of qubits changes from 39 to 40, the trend reverses, and a cluster with 1,024 GPUs has a lower throughput than a cluster with 256 GPUs. The likely cause is the network configuration: Perlmutter GPUs are stored in groups located in different racks, leading to increased communication costs if the rack boundary needs to be crossed. Another potential reason is that Perlmutter jobs are assigned to different GPUs, some of which are not warmed up (resulting in lower efficiency), thereby increasing the circuit simulation time. Therefore, we note that there is an energy trade-off to achieve the best QCS performance by setting the physical hardware in a single territory. Fig. 4c compares the performance of **Q-Gear** and **PennyLane** (using the `lightning.gpu` backend [1]) for QFT circuit simulations on a 4×A100 GPU cluster. **Q-Gear** consistently outperforms **PennyLane**, achieving significantly faster runtimes and showing better scaling with increasing circuit size. We note that containerized **PennyLane** is not optimized for large-scale simulations because the native GNU-distributed interface is not fully utilized when initializing the container in the workflow.

Q-Gear simulations of image encoding

QCrack encoding [2] allows a large grayscale image to be stored in the quantum state. This provides a constructive algorithm for generating a unitary image consisting mainly of Ry rotations and CX gates without variationally adjustable parameters. **QCrack** offers high parallelism in the execution of the CX gate, with the count of the CX gate equal to the number of gray pixels in the input image. A distinguishing feature of **QCrack** is that it not only defines the procedure to recover an image previously stored on a QPU but also allows a meaningful computation of the quantum representation.

For the selected grayscale images listed in Table 2, we first generated **Qiskit** unitaries encoding the images. We ran **Qiskit**

¹The simulations were executed on different types of hardware available at NERSC using various software backends. See Section 2.3 for details on the hardware used.

Table 1: Q-Gear experiments conducted on real CPU/GPU NERSC HPC.

Tasks	Random entangled circuits		QFT transform	Quantum image encoding	
Objective	Speed-up analysis	Scalability analysis	Precision performance	Speed-up analysis	Reconstruction performance
Hardware	32/64-core AMD EPYC NVIDIA A100 HPE Slingshot 11	NVIDIA A100 HPE Slingshot 11	NVIDIA A100 HPE Slingshot 11	64-core AMD EPYC NVIDIA A100 HPE Slingshot 11	
Qubits	28-34	42	16-33	15-25	
Max gate depth	10,000	3,000	528	98,000	
Shots	3,000	10,000	100	3M-98M	
Precision	fp32/fp64	fp32	fp32/fp64	fp64	
Input size	100/10k CX-block	3,000 CX-block	65K-8B bits	5K-98K pixels	

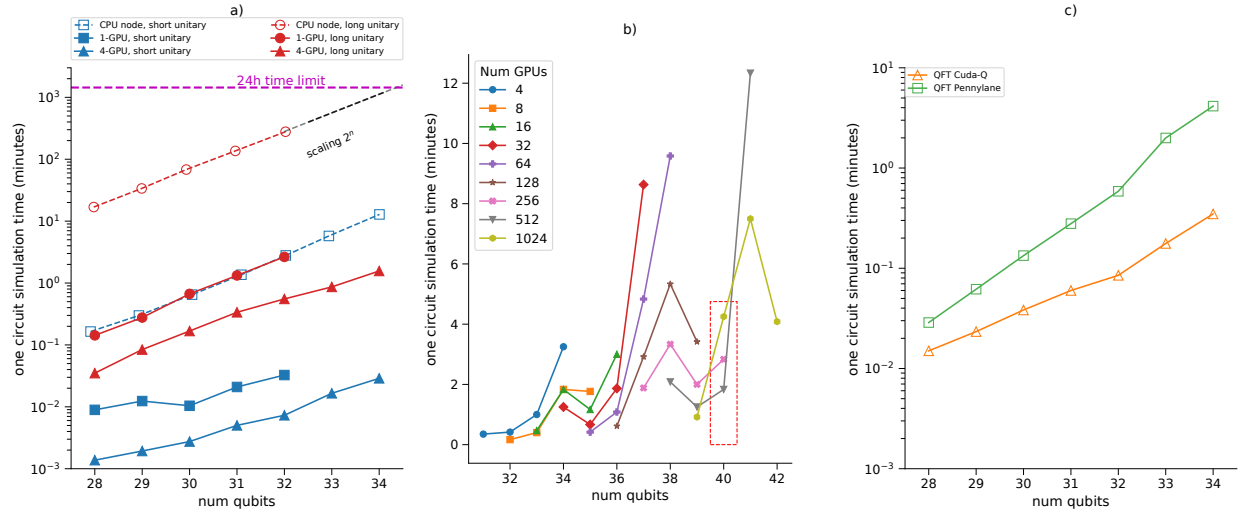


Figure 4: a) Q-Gear simulation speed on a single A100 GPU and on four GPU clusters are shown as solid points. Two sizes of random unitaries, 'short' and 'long' were evaluated, as explained in the text. It is 400x faster vs. the baseline Qiskit performance on a CPU node with 128 cores. b) Scaling for random circuits of 30–42 bits executed on a cluster of 4–1024 A100 GPUs. c) Scaling for the QFT circuit execution time on four A100 GPUs with Q-Gear is compared with native PennyLane execution.

Table 2: Quantum circuit configurations for various gray-scale images. Image dimensions determine qubit usage and shot counts for execution based on $s * 2^m$, where $s = 3000$ is the shot count per address and m is the number of address qubits.

Image	Dims	Gray Px	Addr Q	Data Q	Shots
Finger	64×80	5k	10	5	3M
Shoes	128×128	16k	11	8	6M
Building	192×128	25k	12	6	12M
Zebra	384×256	98k	13	12	24M
Zebra	384×256	98k	14	6	49M
Zebra	384×256	98k	15	3	98M

simulations on one CPU node, as before, to establish the baseline, as shown by the open circles in creffig: qcrank. We then used Q-Gear to convert Qiskit circuits to Cuda-Q circuits and configured Cuda-Q

to use a single A100 GPU. The resulting simulation times are represented by solid circles. See benchmark details in Fig. 6.

The Q-Gear interface results in simulations that are almost two orders of magnitude faster for smaller image sizes. The speedup decreases for larger images, most likely because achieving a similar image recovery fidelity requires more shots for larger ones. In such cases, the total execution time has two components of comparable duration: unitary computation and sampling shots from the unitary. For Qiskit CPU simulations, the unitary was computed independently for each core, whereas sampling was performed in parallel on all 128 CPU cores. Serial sampling was performed for Cuda-Q simulations on a single GPU. Therefore, for a large number of shots, a CPU node with many cores may be advantageous over a single GPU.

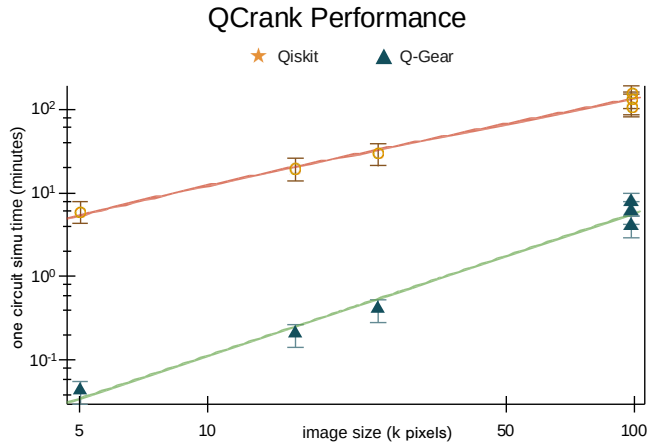


Figure 5: Performance comparison of Qiskit on CPU node and Q-Gear on one A100 GPU for circuits encoding gray-scale images using QCrack. The error bars indicate the observed running time variability (~5%). For both methods the running time scales with the image size because the pixel count is equal to the number of entangling gates in the circuit.

4 Discussion

This study presents two key contributions to quantum circuit migration and execution on HPC platforms. First, the **Q-Gear** framework provides seamless migration of **Qiskit** circuits to **Cuda-Q** kernels, allowing efficient GPU-based simulation using **Cuda-Q**. Second, we built a customized Podman-HPC image with CUDA kernels and CUDA-aware MPI, which was deployed in a public repository at NERSC. Moreover, since Docker and Podman share the same syntax, users familiar with Docker can easily adopt this Podman image without modifying their existing workflows. This compatibility ensures that the images are versatile and practical for a wide range of researchers.

For QFT circuit execution on GPUs, we found that **Q-Gear** using **Cuda-Q** outperforms **Pennylane** despite both leveraging the cuQuantum state vector backends. The primary reason is that when **Pennylane** invokes the **Cuda-Q** 'statevector' backend, the simulation process takes longer because it must first transpile high-level Python representations into low-level CUDA kernels, an additional step that introduces latency. In contrast, directly mapping quantum circuits into CUDA kernels eliminates this overhead, resulting in a more efficient simulation. We note that Qiskit-GPU [16] could not be tested because of specific dependencies on the PyTorch version that were not supported in NERSC. Similarly, Qulacs [35] lacks maintained GPU Python packages, and its complex backend installation process using binary packages deters researchers. We were unable to evaluate TensorFlow Quantum [7] because it relies on deprecated packages, which makes it impractical for current research requirements. These limitations highlight the superior accessibility and performance provided by **Q-Gear** for high-performance quantum simulations on a GPUs. Finally, we note that **Q-Gear** currently operates with **Cuda-Q** with NVIDIA GPUs that rely on cuQuantum libraries [3]², which

²<https://developer.nvidia.com/cuquantum-sdk>

also support AMD GPUs with the flexibility to migrate to other HPCs.

Acknowledgments

This research used resources of the National Energy Research Scientific Computing Center, a DOE Office of Science User Facility supported by the Office of Science of the U.S. Department of Energy under Contract No. DE-AC02-05CH11231 using NERSC award NERSC DDR-ERCAP0034486. We thank Thien Nguyen and Zohim Chandani from NVIDIA for their assistance in running the framework at scale.

References

- [1] Ali Asadi, Amintor Dusko, Chae-Yeun Park, Vincent Michaud-Rioux, Isidor Schoch, Shuli Shu, Trevor Vincent, and Lee James O'Riordan. 2024. Hybrid quantum programming with PennyLane Lightning on HPC platforms. arXiv:2403.02512 [quant-ph] <https://arxiv.org/abs/2403.02512>
- [2] J. Balewski, M.G. Amankwah, R. Van Beeumen, et al. 2024. Quantum-parallel vectorized data encodings and computations on trapped-ion and transmon QPUs. *Nature Scientific Reports* 14 (2024), 3435. doi:10.1038/s41598-024-53720-x
- [3] Harun Bayraktar, Ali Charara, David Clark, Saul Cohen, Timothy Costa, Yao-Lung L. Fang, Yang Gao, Jack Guan, John Gunnels, Azzam Haidar, et al. 2023. cuQuantum SDK: A High-Performance Library for Accelerating Quantum Science. In *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, Vol. 1. IEEE, Los Alamitos, CA, USA, 1050–1061.
- [4] Elizabeth C. Behrman et al. 2000. Simulations of quantum neural networks. *Information Sciences* 128, 3–4 (2000), 257–269.
- [5] Jacob Biamonte et al. 2017. Quantum machine learning. *Nature* 549, 7671 (2017), 195–202.
- [6] Lennart Bittel and Martin Kliesch. 2021. Training variational quantum algorithms is NP-hard. *Physical Review Letters* 127, 12 (2021), 120502.
- [7] Michael Broughton, Guillaume Verdon, Trevor McCourt, Antonio J. Martinez, Jae Hyeon Yoo, Sergei V. Isakov, Philip Massey, Ramin Halavati, Murphy Yuezheng Niu, Alexander Zlokapa, Evan Peters, Owen Lockwood, Andrea Skolik, Sofiene Jerbi, Vedran Dunjko, Martin Leib, Michael Streif, David Von Dollen, Hongxiang Chen, Shuxiang Cao, Roeland Wiersema, Hsin-Yuan Huang, Jarrod R. McClean, Ryan Babbush, Sergio Boixo, Dave Bacon, Alan K. Ho, Hartmut Neven, and Masoud Mohseni. 2021. TensorFlow Quantum: A Software Framework for Quantum Machine Learning. arXiv:2003.02989 [quant-ph] <https://arxiv.org/abs/2003.02989>
- [8] Marco Cerezo et al. 2021. Variational quantum algorithms. *Nature Reviews Physics* 3, 9 (2021), 625–644.
- [9] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. Nvidia a100 tensor core gpu: Performance and innovation. *IEEE Micro* 41, 2 (2021), 29–35.
- [10] Andrew W Cross, Easwar Magesan, Lev S Bishop, John A Smolin, and Jay M Gambetta. 2016. Scalable randomised benchmarking of non-Clifford gates. *npj Quantum Information* 2, 1 (2016), 1–5.
- [11] Amit Jamadagni Gangapuram, Andreas Läuchli, and Cornelius Hempel. 2024. Benchmarking quantum computer simulation software packages: State vector simulators. *SciPost Physics Core* 7, 4 (2024), 075.
- [12] Lisa Gerhardt, Wahid Bhimji, Shane Canon, Markus Fasel, Doug Jacobsen, Mustafa Mustafa, Jeff Porter, and Vakho Tsulaia. 2017. Shifter: Containers for hpc. In *Journal of physics: Conference series*, Vol. 898. IOP Publishing, Bristol, England, 082021.
- [13] William Gropp. 2002. MPICH2: A new start for MPI implementations. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface: 9th European PVM/MPI Users' Group Meeting Linz, Austria, September 29–October 2, 2002 Proceedings* 9. Springer, ACM, Linz, Austria, 7–7.
- [14] Lov K Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*. ACM, Philadelphia, Pennsylvania, USA, 212–219.
- [15] Hiroshi Horii, Christopher Wood, et al. 2023. Efficient Techniques to GPU Accelerations of Multi-Shot Quantum Computing Simulations. *arXiv preprint abs/2308.03399* (2023), 15 pages. arXiv:2308.03399 <https://arxiv.org/abs/2308.03399>
- [16] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. 2024. Quantum computing with Qiskit. arXiv:2405.08810 [quant-ph] doi:10.48550/arXiv.2405.08810
- [17] S. K. Jeswal and S. Chakraverty. 2019. Recent developments and applications in quantum neural network: A review. *Archives of Computational Methods in Engineering* 26, 4 (2019), 793–807.

- [18] Morris A. Jette and Tim Wickberg. 2023. Architecture of the Slurm workload manager. In *Workshop on Job Scheduling Strategies for Parallel Processing*. Springer Nature Switzerland, St. Petersburg, FL, USA, 1–10.
- [19] Chenyang Jiao, Weihua Zhang, and Li Shen. 2023. Communication optimizations for state-vector quantum simulator on CPU+ GPU clusters. In *Proceedings of the 52nd International Conference on Parallel Processing*. ACM, New York, NY, USA, 203–212.
- [20] B. Keimer and J. E. Moore. 2017. The physics of quantum materials. *Nature Physics* 13, 11 (2017), 1045–1055.
- [21] Jin-Sung Kim et al. 2023. Cuda quantum: The platform for integrated quantum-classical computing. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, San Francisco, CA, USA, 1–6.
- [22] Sandeep Koranne. 2010. Hierarchical Data Format 5: HDF5. In *Handbook of Open Source Tools*. Springer, Boston, MA, USA, 191–200.
- [23] Phuc Q Le, Fangyan Dong, and Kaoru Hirota. 2011. A flexible representation of quantum images for polynomial preparation, image compression, and processing operations. *Quantum Information Processing* 10 (2011), 63–84.
- [24] Michael Lubasch et al. 2020. Variational quantum algorithms for nonlinear problems. *Physical Review A* 101, 1 (2020), 010301.
- [25] David C. McKay et al. 2018. Qiskit Backend Specifications for OpenQASM and OpenPulse Experiments. *arXiv preprint abs/1809.03452* (2018), 17 pages. <https://arxiv.org/abs/1809.03452>
- [26] L. Moro, M.G.A. Paris, M. Restelli, et al. 2021. Quantum compiling by deep reinforcement learning. *Communications Physics* 4 (2021), 178. doi:10.1038/s42005-021-00684-3
- [27] M. Möttönen, J. J. Vartiainen, V. Bergholm, and M. M. Salomaa. 2004. Quantum circuits for general multiqubit gates. *Physical Review Letters* 93, 13 (2004), 130502. doi:10.1103/PhysRevLett.93.130502
- [28] NVIDIA. 2025. NVIDIA Nsight Systems. Computer software, version 2025.3.1. <https://developer.nvidia.com/nsight-systems> Accessed 10 Jul 2025.
- [29] Mateusz Ostaszewski et al. 2021. Reinforcement learning for optimization of variational quantum circuit architectures. *Advances in Neural Information Processing Systems* 34 (2021), 18182–18194.
- [30] Ziwen Pan et al. 2020. Secret-key distillation across a quantum wiretap channel under restricted eavesdropping. *Physical Review Applied* 14, 2 (2020), 024044.
- [31] David Pérez-García, Frank Verstraete, Michael M. Wolf, and J. Ignacio Cirac. 2006. Matrix Product State Representations. *arXiv preprint abs/quant-ph/0608197* (2006), 38 pages. <https://arxiv.org/abs/quant-ph/0608197>
- [32] Maria Schuld, Alex Bocharov, Krysta M. Svore, and Nathan Wiebe. 2020. Circuit-Centric Quantum Classifiers. *Physical Review A* 101, 3, Article 032308 (March 2020), 15 pages. doi:10.1103/PhysRevA.101.032308
- [33] Peter W Shor. 1994. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th annual symposium on foundations of computer science*. IEEE, Santa Fe, New Mexico, USA, 124–134.
- [34] Laurie Stephey et al. 2022. Scaling Podman on Perlmutter: Embracing a community-supported container ecosystem. In *2022 IEEE/ACM 4th International Workshop on Containers and New Orchestration Paradigms for Isolated Environments in HPC (CANOPIE-HPC)*. IEEE, Dallas, Texas, USA, 41–50.
- [35] Yasunari Suzuki, Yoshiaki Kawase, Yuya Masumura, Yuria Hiraga, Masahiro Nakadai, Jiabao Chen, Ken M. Nakanishi, Kosuke Mitarai, Ryosuke Imai, Shiro Tamaiya, Takahiro Yamamoto, Tennin Yan, Toru Kawakubo, Yuya O. Nakagawa, Yohei Ibe, Youyuan Zhang, Hirotsugu Yamashita, Hikaru Yoshimura, Akihiro Hayashi, and Keisuke Fujii. 2021. Qulacs: a fast and versatile quantum circuit simulator for research purpose. *Quantum* 5 (Oct. 2021), 559. doi:10.22331/q-2021-10-06-559
- [36] Kai Troester and Ravi Bhargava. 2023. AMD Next Generation “Zen 4” Core and 4th Gen AMD EPYC™ 9004 Server CPU. In *2023 IEEE Hot Chips 35 Symposium (HCS)*. IEEE Computer Society, IEEE, Palo Alto, CA, USA, 1–25.
- [37] Ruben Van Beeumen, Dries Camps, and Neil Mehta. 2023. QCLAB++: Simulating Quantum Circuits on GPUs. *arXiv:2303.00123 [quant-ph]* <https://arxiv.org/abs/2303.00123>
- [38] Yaakov S. Weinstein et al. 2001. Implementation of the quantum Fourier transform. *Physical Review Letters* 86, 9 (2001), 1889.
- [39] Y.-H. Zhang, P.-L. Zheng, Y. Zhang, and D.-L. Deng. 2020. Topological Quantum Compiling with Reinforcement Learning. *Physical Review Letters* 125, 17 (2020), 170501. doi:10.1103/PhysRevLett.125.170501
- [40] Jan Balewski Ziqing Guo and Ziwen Pan. 2025. Q-Gear. <https://github.com/gzquse/Q-Gear>. Version: v0.0.1; accessed 20 Jul. 2025.

A Quantum state vector simulation

In this section, we detail the procedure for encoding quantum gates in the statevector simulator, which can then be used for sampling using a user-specified number of shot. Inspired by quantum simulations with the CUDA parallelizable programming model [19], we transformed

the gates into discrete tensors and then into a sequence of kernels, as shown in Fig. 2(a). Specifically, a statevector simulator maintains an n -qubit register as a length- 2^n complex vector and updates it by multiplying it with the corresponding $2^n \times 2^n$ unitaries. In the computational basis this register is written as

$$|\psi\rangle = \sum_{i=0}^{2^n-1} \alpha_i |i\rangle, \quad \alpha_i \in \mathbb{C}, \quad \sum_{i=0}^{2^n-1} |\alpha_i|^2 = 1, \quad (1)$$

where α_i is the complex amplitude. Throughout our experiments, we employed the single-qubit gates R_x and R_y together with the two-qubit controlled- X (CX) gate. Embedding a single-qubit unitary U on the k -th qubit is achieved with

$$U^{(k)} = I^{\otimes(k-1)} \otimes U \otimes I^{\otimes(n-k)}, \quad (2)$$

which is a $2^n \times 2^n$ matrix. Note that CX gate with control qubit k and target qubit m ($k \neq m$) via the computational-basis projectors $\Pi_0 = |0\rangle\langle 0|$ and $\Pi_1 = |1\rangle\langle 1|$

$$U_{CX}^{(k \rightarrow m)} = \Pi_0^{(k)} + \Pi_1^{(k)} X^{(m)}, \quad X^{(m)} = I^{\otimes(m-1)} \otimes X \otimes I^{\otimes(n-m)}, \quad (3)$$

thereby $U_{CX}^{(k \rightarrow m)}$ is a well-defined $2^n \times 2^n$ unitary for every n , k , and m , eliminating any occurrence of negative tensor exponents. Referring to Fig. 4a, the GPU simulation runs time scales linearly with an increase in the CX gate tensor product operation because of the parallelism of the GPU architectures. Detailedly, CUDA-enabled GPUs consist of thousands of cores organized into streaming multiprocessors (SMs), each capable of executing multiple threads concurrently. These threads are grouped into warps (32 threads) that execute instructions in lockstep. Therefore, **Q-Gear** achieves state-vector simulation parallelism by distributing independent Podman containers across these threads, allowing the simultaneous execution of different quantum circuits.

In this section, we present an example of a qubit system. Let us consider a 3-qubit state vector

$$|\psi\rangle = \alpha_{000}|000\rangle + \alpha_{001}|001\rangle + \cdots + \alpha_{111}|111\rangle. \quad (4)$$

If the control qubit is q_0 (first qubit) and the target is q_2 (third qubit), we identify all the basis states, where $q_0 = 1: |100\rangle, |101\rangle, |110\rangle, |111\rangle$. In these states, the swap amplitudes for q_2 are $\alpha_{100} \leftrightarrow \alpha_{101}$ and $\alpha_{110} \leftrightarrow \alpha_{111}$. The CX gate involves non-contiguous memory access because the amplitudes to be swapped are scattered across the state vector. Each CX gate modifies 2^{n-1} amplitudes in the state vector with the number of operations scaled as $O(2^n \cdot d)$, where d is the number of CX blocks. Consequently, the workflow can achieve a linear speedup by distributing quantum circuits across multiple GPUs, each processing a subset of the state vector.

B High dimensional data management

To enhance the efficiency of managing high-dimensional datasets, we employed the HDF5 file format, which supports the following three properties:

- (1) **Hierarchical Data Storage:** efficient organization of tensors, circuits, and metadata.
- (2) **Scalability:** seamless handling of large datasets, reducing read/write overhead.
- (3) **Compression:** storage efficiency through lossless data compression.

Using the HDF5 format, the encoding and time-saving complexity is $O(N \cdot T \cdot \log(T))$, with the logarithmic factor accounting for optimized indexing. Our implementation shows that for fixed tensor sizes (T), the encoding time remains nearly constant regardless of the circuit complexity. For example, encoding $N = 1000$ circuits with $T = 10^6$ took 2 min, independent of entanglement depth or gate count. Furthermore, HDF5 compression reduced storage by up to 50% without affecting read/write speeds, demonstrating its efficiency for large-scale quantum datasets.

C Circuit encoding and tensor transformation

LEMMA C.1. *Any quantum gate acting on n qubits can be represented as a vector in 2^{2n} -dimensional space, where the vector encodes the gate unitary matrix elements.*

Proof: Let U be a quantum gate acting on n qubits. The gate U is represented by a unitary matrix of size $2^n \times 2^n$, where each element of the matrix corresponds to a complex number. To encode this matrix as a vector, we flattened the matrix into a single-column vector of size 2^{2n} , where the elements are ordered row by row (or column by column, depending on the convention).

For a single-qubit gate U_k acting on the k -th qubit, the operation can be expressed by Eq. (2), where I is the identity matrix acting on the unaffected qubits, and U_k is the gate operation on the k -th qubit. This directly gives us a 2^{2n} flattened vector. Similarly, for a multiqubit gate U_{ij} (e.g., controlled gates), the unitary matrix applies transformations that depend on both control and target qubits, as shown in Eq. (3). Matrix U_{ij} can also be flattened into a vector of size 2^{2n} because the total number of elements remains the same, and this transformation preserves all the information about the matrix, unlike the matrix product state [31].

LEMMA C.2. *For any quantum circuit, a fixed-size tensor can represent the gates without loss of information provided that the tensor size d is satisfied through Eq. (5).*

$$d \geq \max(|G|, |C|), \quad (5)$$

where $|G|$ is the total number of gates in the largest circuit and $|C|$ is the number of circuits encoded.

Proof: By definition, the tensor dimensions in our pipeline are determined before processing, ensuring sufficient capacity to store circuit data. During the encoding step, the tensor is initialized to its maximum allowable size d and iteratively overridden as the circuits are processed. Given a uniform encoding strategy, the tensor length scales linearly with the number of circuits, thereby ensuring that no truncation or overflow occurs. We now prove the universality theorem.

THEOREM C.3. *For a quantum circuit simulation task with N qubits (or gates), the computational time t scales exponentially on a CPU but linearly on a GPU using containerized submission, i.e.,*

$$t_{CPU}(N) \sim O(2^N) \quad \text{and} \quad t_{GPU}(N) \sim O(N) \quad (6)$$

Proof: The quantum circuit state vector for N qubits is represented in 2^N -dimensional Hilbert space. Given the sequential nature of CPUs, each gate operation requires $O(2^{2N})$ operations in the worst case because of the $2^N \times 2^N$ matrix representation, as shown by

Lemma C.1. Processing quantum gates or encoding such vectors involves sequential multiplications of unitary matrices in CPUs, followed by Lemma C.2.

We denote that the **Q-Gear** transformation exploits parallelism by simultaneously applying operations to independent vector components inside each **Podman** container. NVIDIA A100 can partition the 2^{2N} -dimensional state space into P parallel cores, where P grows proportionally to the available hardware resources and the state vector initialization becomes trivial. Consequently, the time complexity per gate was reduced to $O(P)$. Given that P is large and typically scales with 2^{2N} , the effective computational complexity for GPUs becomes:

$$t_{GPU}(N) \sim O(N). \quad (7)$$

Consequently, while CPU computation time grows exponentially, GPU computation time scales linearly with the problem size N , provided that sufficient parallel resources are available.

C.1 Shifter Multiple Nodes

In the multi-node mode **Q-Gear**, we utilize the **Cuda-Q** nightly to build a Shifter image and predefined essential tools (e.g., **qiskit-aer**, **h5py**, **qiskit-ibm-experiment**, etc.) in the local scratch file system to ensure a consistent environment for all submitted jobs.

The architecture leverages Shifter for multinode-scalable CUDA-enabled quantum simulations using **Cuda-Q** within a containerized setup, as illustrated in Appendix D.1. The difference is that the workloads are distributed across different physical nodes via MPI (cray mpich), enabling the parallel execution of complex and highly entangled quantum circuits. Theoretically, the upper bound is defined by the allocated resources with time-complexity scaling, as shown in Eq. (8).

$$t_{\text{multi-node}} \sim O\left(\frac{2^N}{P \cdot R}\right), \quad (8)$$

where N denotes the number of qubits, P denotes the number of parallel processes per node, and R denotes the number of nodes, respectively. We also note that the pipeline dynamically detects the GPUs available at each node, ensuring full utilization by running threads. Inter-node communication is managed through NVLink broadcasts, which facilitate the sharing of information across tasks. In addition, our design supports diverse workloads with different container images, enabling large-scale quantum circuit benchmarking and high-throughput quantum image processing for further research.

D Pipeline Configuration

In this section, we present the details of the two deployment modes: the Podman-HPC container mode and Shifter [12] scaling node mode. Note that the difference between Podman-HPC and Shifter is that Shifter condenses the Podman images into a 'squashed' format, which is easier for scaling. In addition, we describe how these modes are used in conjunction with the **Q-Gear** transformation.

D.1 Podman HPC Container

The Podman container includes essential dependencies such as **copy-cuda12x**, **mpi4py**, **Qiskit**, and **Cuda-Q**, along with libraries for visualization and statevector simulation. To create a Podman container, we use a pre-installed **cu12.0** DevOps container as the

base image and integrate the native NERSC Cray MPICH, which is optimized for high-performance interconnects and parallel processing in NERSC systems.

To streamline the workflow and maximize resource utilization, we developed a robust Slurm job submission framework that seamlessly integrates MPI parallelization (Fig. 2(c)), empowering users to configure key parameters, such as the number of random circuits, simulation shots, QFT circuit reverse activation, and transformation precision. We established a Shell script, built for the containerized environment, which employs a novel technique dubbed "podman wrapper" that dynamically links batch submission variables, environment parameters (e.g., MPI rank), locally generated circuits, and output directories to the containerized execution environment. By leveraging the workflow, efficient resource orchestration and precise data management are ensured throughout the simulation pipeline. Note that the performance metrics captured by the NVIDIA GPU performance monitoring NSight system [28] demonstrate significant improvements in computational efficiency and scalability. The complete codebase and resources are available on the GitHub for further exploration and reproducibility.

E Datasets

In this section, we describe the procedure for generating three types of datasets with which **Q-Gear** can uniformly perform the transformation using the same structured gate lists.

E.1 Random circuit generator (CX-block)

We define a method, `generate_random_gateList`, that generates a randomized list of quantum circuits characterized by specific gate types and configurations. Each circuit is described by the number of qubits, specific target and control qubit indices, type of gates, and input parameters for variational unitary quantum gates [8]. More specifically, the generator function pre-allocates the CUDA memory for circuit layout, for instance, we implement a numpy two-dimensional array for gate types $M = (h, ry, rz, cx, \text{measure})$ mapping to one hot encoding for optimization shown in Eq. (9).

$$M^T = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (9)$$

We then implemented block circuits with a sequential fixed number of CX gates and interleaved them with randomly paired parameterized R_y and R_z rotations to ensure alternating qubit pairings. The applied algorithm is described in Algorithm 1.

The CX block output includes structured arrays for circuit properties (`circ_type`), gate specifications (`gate_type`), and gate parameters (`gate_param`), along with metadata for the circuit generation process. The random CX block function generates a specified number of random qubit pairs from a given number of qubits. For example, (1,2) means that the first control qubit targets the second qubit. This block constructs all possible ordered pairs of qubits, excluding self-pairs (i.e., pairs in which a qubit is paired). The function then randomly selects k pairs with replacements from the set of valid pairs returned as a 2D network array.

Algorithm 1 Randomized Quantum Circuit Generation

```

1: Input: Quantum circuit  $C$ 
2: Let  $Q$  be the set of qubits in  $C$ 
3: for each gate  $G$  in  $C$  do
4:   Sample  $q_c \in Q$ 
5:   repeat
6:     Sample  $q_t \in Q$ 
7:   until  $q_t \neq q_c$ 
8:   Assign gate parameters  $\theta \sim \mathcal{U}[0, 2\pi]$  for randomized rotation gates
9: end for

```

E.2 QFT kernel generator

We customized the QFT kernel starting with the Hadamard gate layer and interconnected controlled arbitrary rotation gates, namely $cr1$, where each gate accepts the flattened parameters transferred from the structured **Qiskit** circuits output tensor-like data [38]. For a quantum register of size n , the kernel applies a Hadamard gate to each qubit followed by Eq. (10).

$$CR1(\lambda) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\lambda} \end{bmatrix} \quad (10)$$

between each qubit i and all subsequent qubits $j > i$, with angles decreasing as $\frac{2\pi}{2^{j-i+1}}$. This nested loop structure introduces only $O(n^2)$ complexity. We then utilized `cuda.qview` to facilitate efficient state manipulation and execution on the NERSC platforms, making it scalable for quantum simulations. To further optimize the kernel, we specify hyperparameters (gate fusion = 5) and approximations for negligible rotation angles to reduce the execution overhead without a significant loss of fidelity, ensuring the applicability of the kernel in **Q-Gear**.

E.3 QCrack circuit generator

We created a utility method for generating preprocessed grayscale image data and metadata for **QCrack**, a quantum simulation, or a computation framework. This method includes image handling, metadata construction, and preparation of quantum input formats adapted for parallel processing and **Q-Gear** kernel transformation.

Our method normalizes grayscale images to $[-1, 1]$ and encodes them using trigonometric transformations to satisfy **QCrack** input requirements. User-configurable options allow control over the image types, output paths, and addressing qubits via command-line arguments. Metadata - covering image dimensions, quantum qubits, and circuit constraints — ensures GPU compatibility. The outputs are structured as $n_{\text{addr}} \times n_{\text{data}} \times n_{\text{img}}$ tensors, with reversed addressing qubits according to **QCrack** conventions, and saved as `.h5` files.

F QCrack Benchmark details

In this study, we present the details of the **QCrack** benchmark utilizing **Q-Gear**. The findings demonstrate that we have encoded all images with less than 0.02 standard deviation, ranging from 5 K to 98 K pixels, using quantum circuits simulated on GPUs.

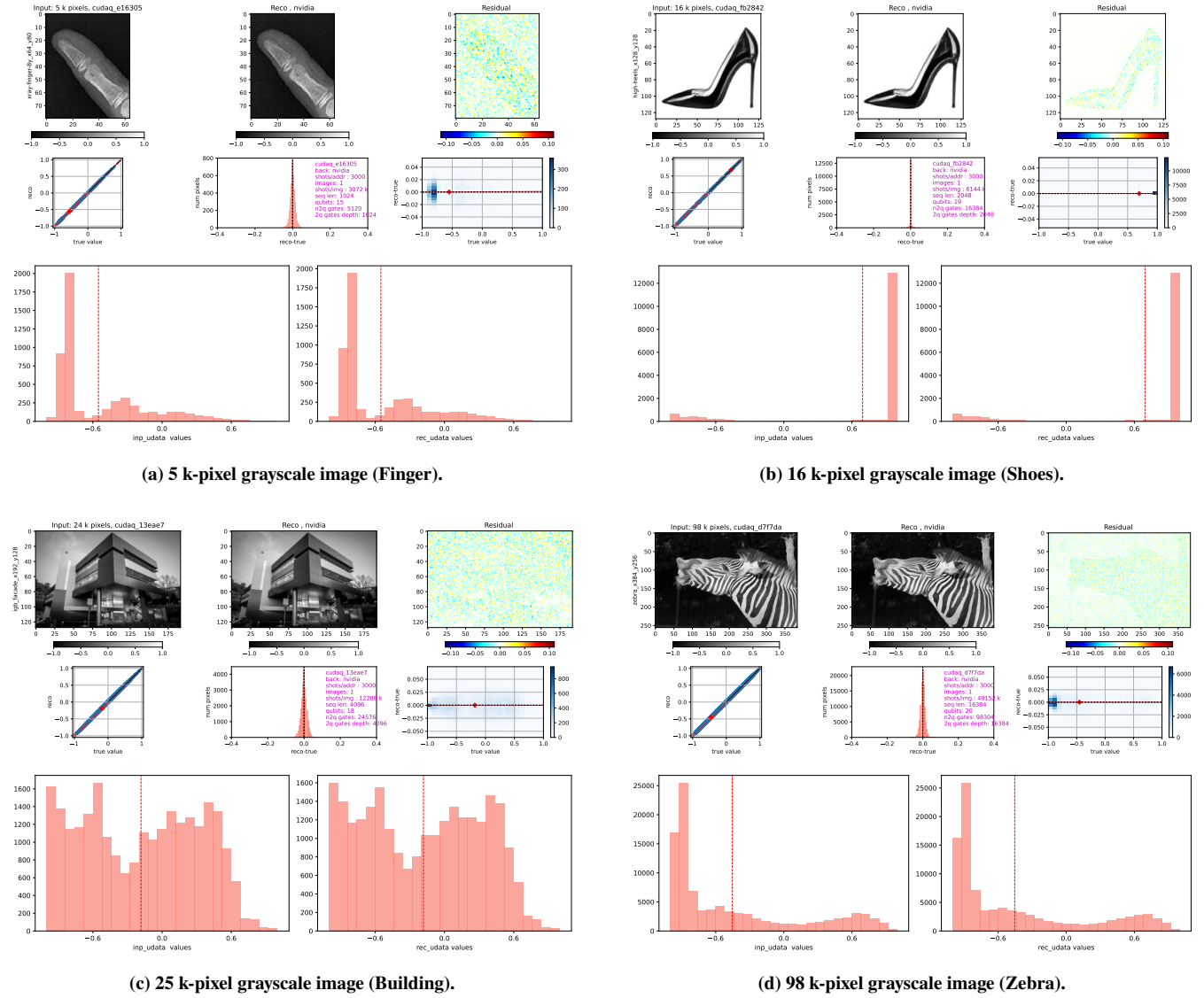


Figure 6: Quantum imageencoding and reconstruction results for four grayscale images with different resolutions. In each sub-figure the top panel shows the reconstruction correlation, the reconstruction distribution, and the residual encoding error, whereas the bottom panel presents the overall reconstruction benchmark.